OrderFlow — Distributed Cache Architecture

Full experimental campaign · 2026-10-11 · campaign_20261011_203001 · Quantitative & qualitative evidence review

Executive conclusion

Decision: REVISE — adopt Redis only for read projections with an explicit freshness contract; do not adopt the current full design as production-ready. Warm cache-aside handles 150 RPS with read p95 2.39 ms, versus 52.6 RPS and 11,123 ms in the deliberately expensive DB-only baseline. This is a strong workload-specific benefit, but Redis outage/restart causes major service degradation, and write-heavy scenarios expose severe variation between strategies. Naive cache-aside produces stale reads (60/60) after updates; L1/L2 also serves stale revisions (24/60).

Evidence base: 52 rows in comparison.csv: 45 load/variant entries and 7 consistency checks (E17); 150 target RPS; 25 s per load; one repetition; 2 application replicas by default; 250,011 activity rows at campaign start. These measurements establish comparative behavior at the tested rate, not maximum sustainable capacity or statistical confidence.

1. Headline performance

Test	Achieved RPS	Read p95	HTTP errors	Dropped
E01 db	52.6	11,122.5 ms	46.9%	1,871
E02 aside	150.0	3.8 ms	0.0%	0
E03 aside	150.0	2.4 ms	0.0%	0
E10 invalidate	74.0	11,055.0 ms	20.8%	1,202
E11 invalidate	84.3	10,812.9 ms	18.9%	1,057
E12a aside	150.0	8.6 ms	0.0%	0
E12b protected	150.0	5.6 ms	0.0%	0
E18a aside	150.0	1.5 ms	0.0%	0
E18b l1l2	150.0	3.1 ms	0.0%	0

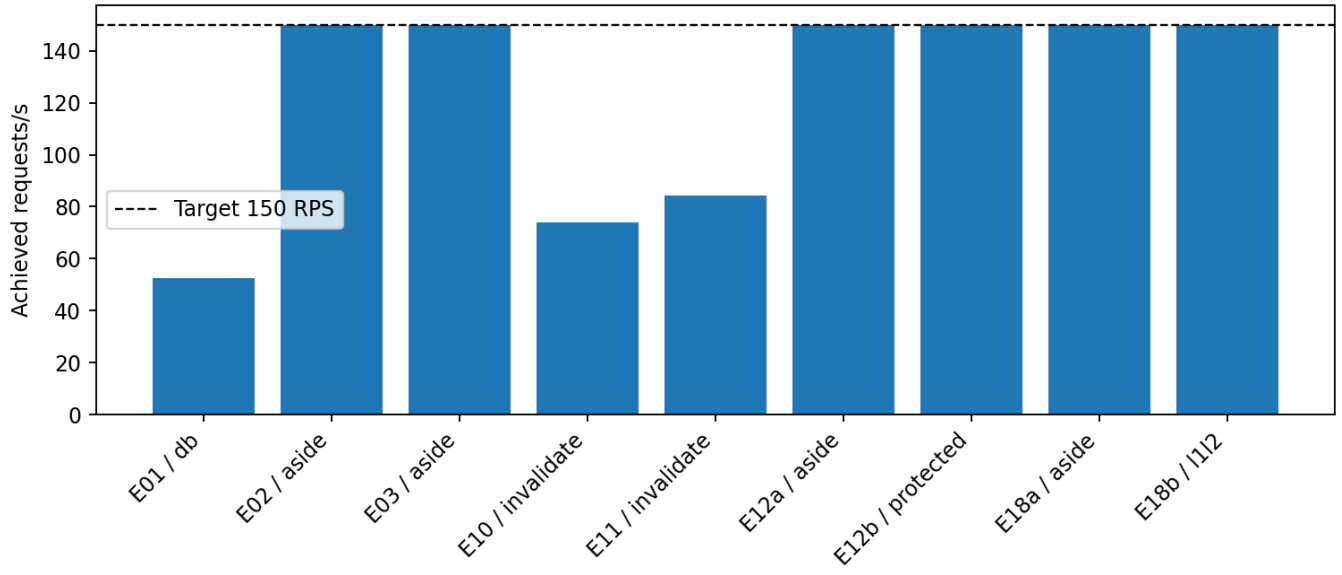


Figure 1. Achieved throughput. A request rate matching the target is not a measured capacity ceiling.

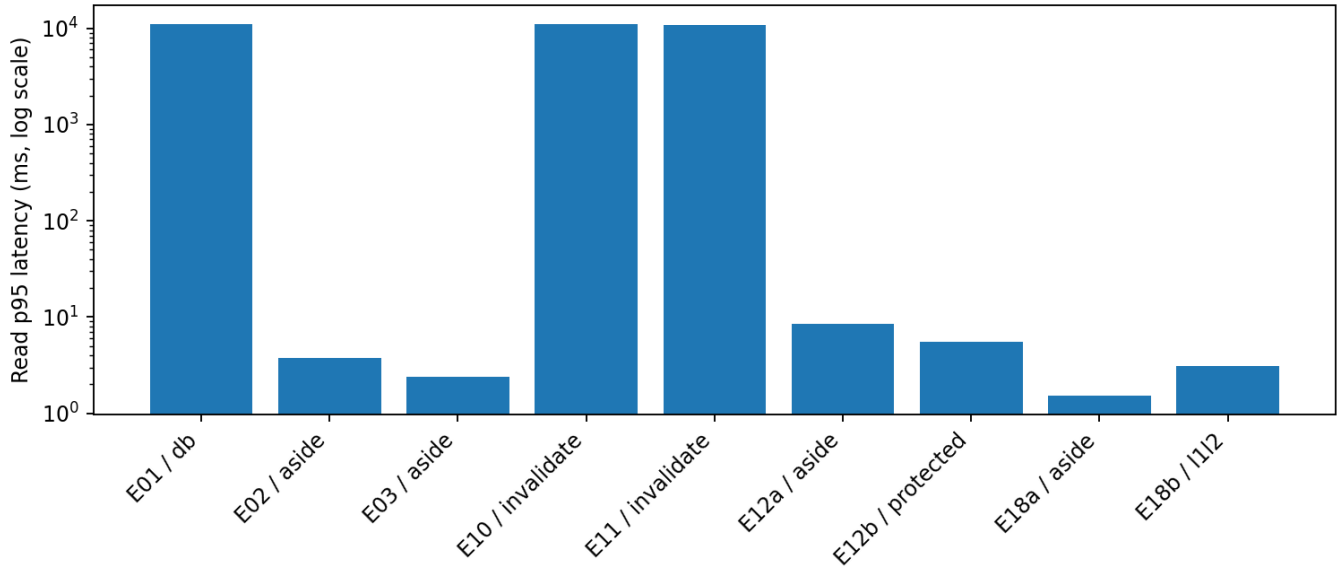


Figure 2. Read p95 latency on a logarithmic scale. DB-only and outage cases include substantial failures and scheduling losses.

2. Write-mix sensitivity (E19)

At 5%, 20%, and 50% writes, basic aside remains near 150 RPS, but this is **not a valid correctness recommendation** because the E17 consistency probe demonstrates stale reads. Write-aware invalidate falls from 128.6 to 103.8 RPS with read p95 increasing from 2.63 s to 6.56 s. Protected stays near 150 RPS, although read p95 increases from 136 to 586 ms. Write-through drops from 149.3 to 79.3 RPS by 50% writes, with 936 dropped iterations and nonzero HTTP errors. Sharding without additional capacity proof offers no clear benefit.

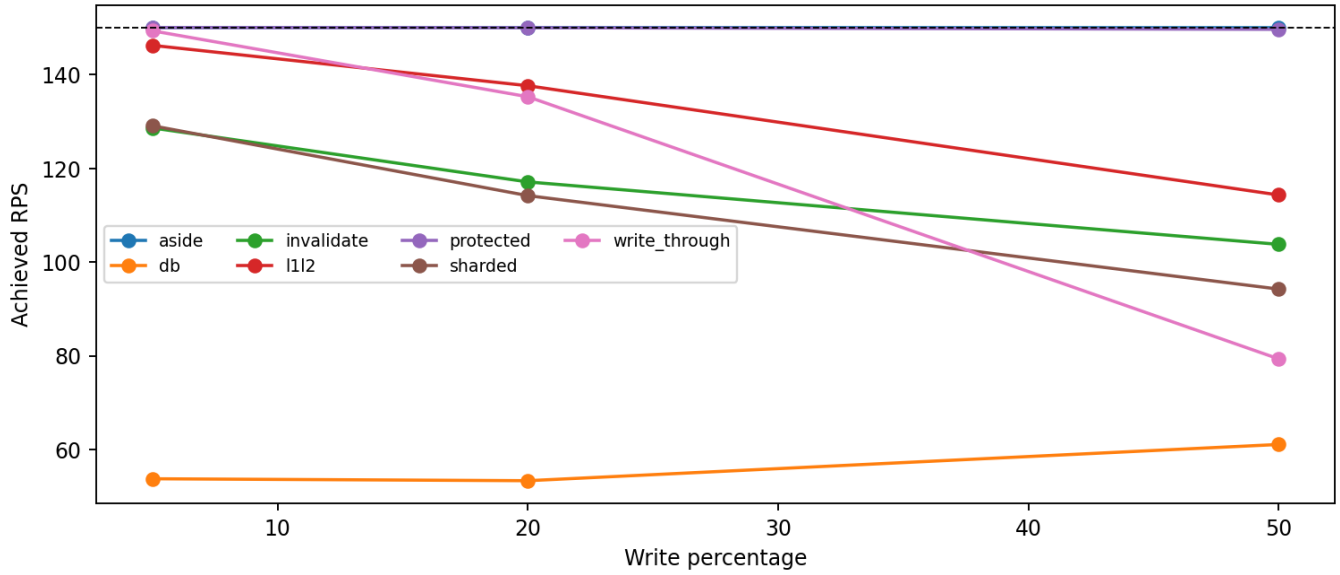


Figure 3. Achieved throughput by write share. Each point is a single campaign run.

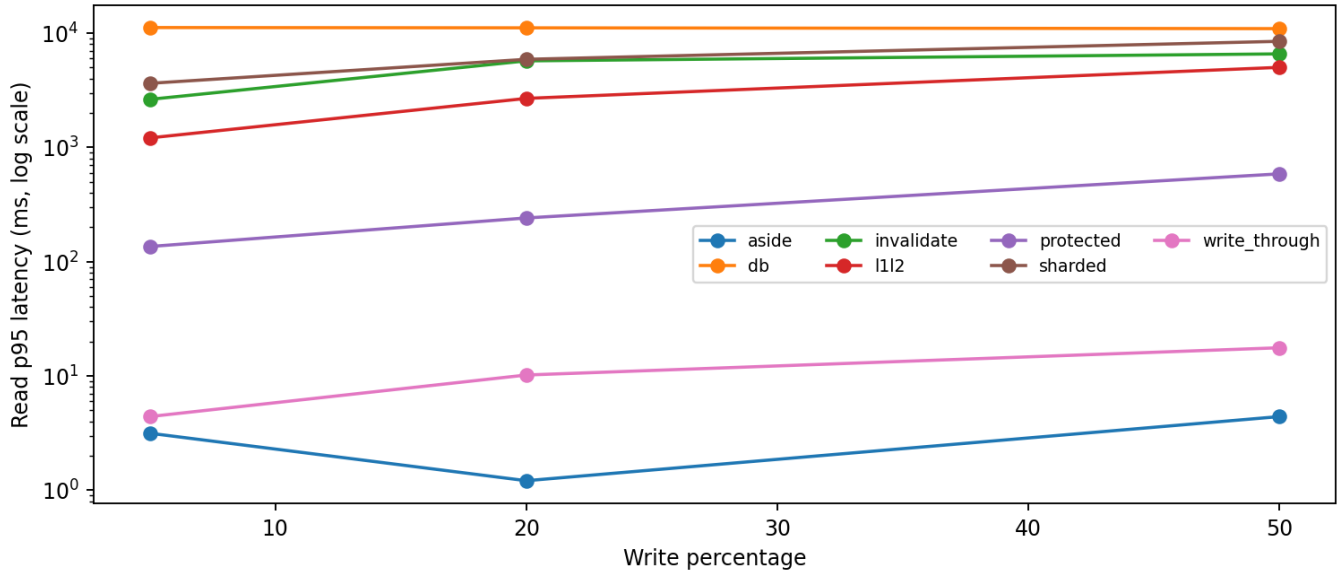


Figure 4. Read p95 by write share. Logarithmic scale makes the multi-order-of-magnitude differences legible.

3. Cache consistency (E17)

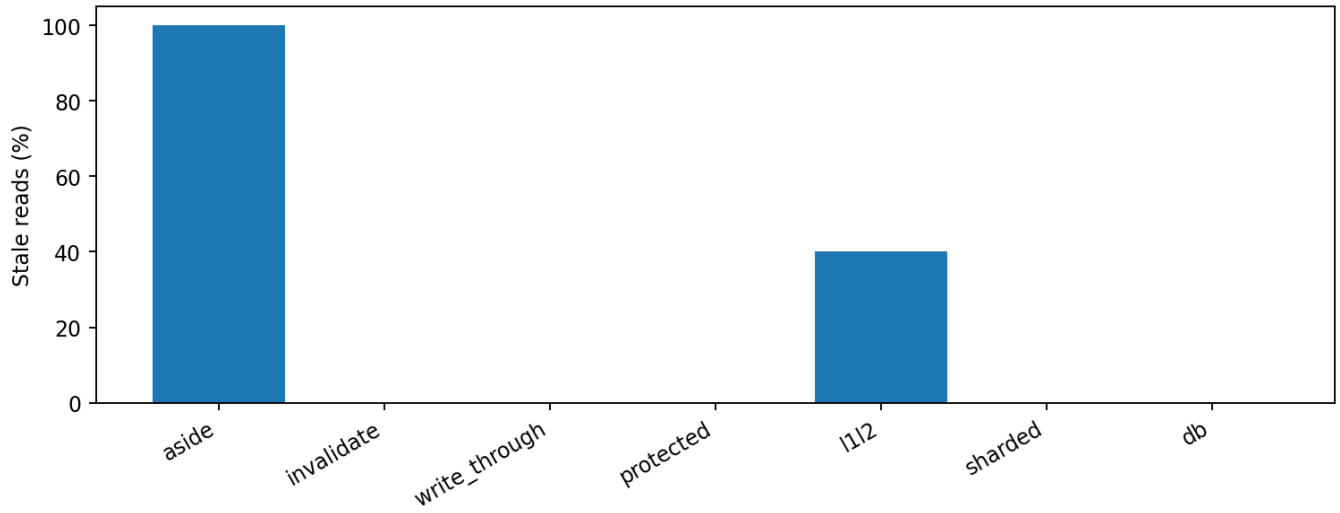


Figure 5. Stale read fraction in sequential probes. These checks are not concurrent consistency proofs.

Basic aside: 60/60 stale reads; L1/L2: 24/60 stale reads. Invalidate, protected, write-through, sharded, and DB returned 0/60 stale reads in this particular probe. This does not guarantee linearizability or absence of race conditions under concurrent readers/writers. Transactional inventory decisions must use the authoritative database independently of the cache.

4. Fault response and recovery (E10–E11)

Redis outage (E10): 74.0 RPS, read p95 11,055 ms, HTTP errors 20.8%, 1,202 dropped iterations. Restart (E11): 84.3 RPS, p95 10,813 ms, HTTP errors 18.9%, 1,057 dropped. These figures reject any claim of acceptable graceful degradation at 150 RPS. Exact recovery time requires aligning raw request timestamps with fault start/stop times and validating sustained restoration of the pre-fault SLO.

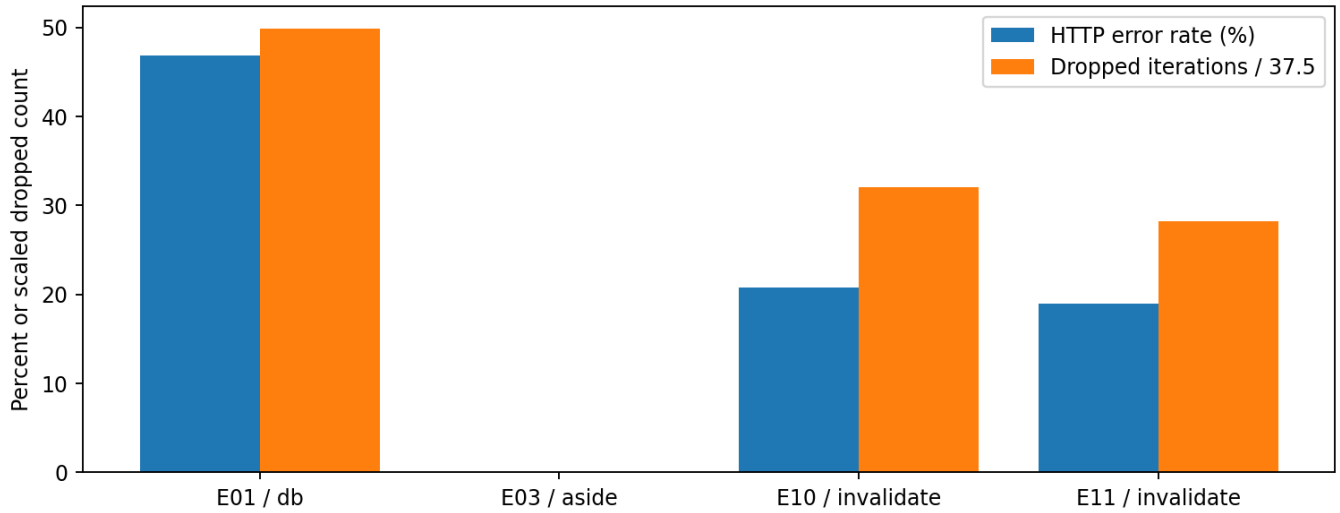


Figure 6. Failure effects; dropped counts are divided by 37.5 solely to share a plotting axis with error percentage (not a percentage metric).

5. Cache TTL and stampede

TTL sensitivity: 1 s shows 857.7 ms p95; 10 s, 11.0 ms; and the longer TTL variant, 2.79 ms. Short TTL induces frequent expensive recomputation, so the throughput and p95 should be evaluated against staleness requirements, not optimized in isolation.

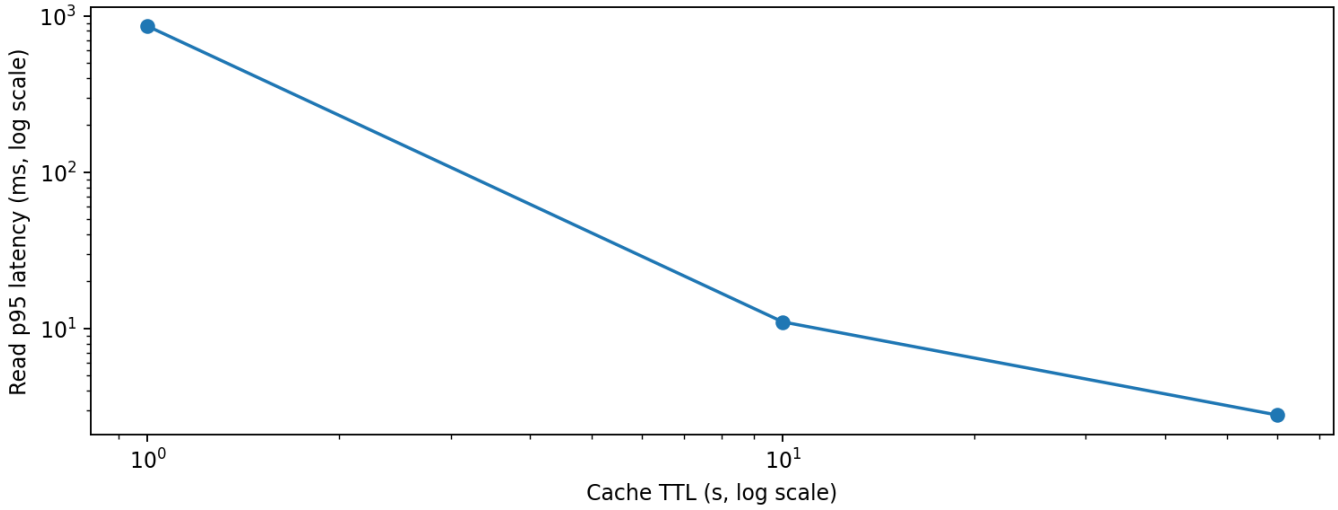


Figure 7. TTL affects the balance between freshness and recomputation cost.

Under a forced cache flush, protected mode reduces observed database-read counter deltas relative to unprotected aside (E12 and E16), consistent with request coalescing. Prometheus-derived deltas can be sampled or incomplete and must be corroborated with DB-level query counters and raw time-series before claiming an exact savings percentage.

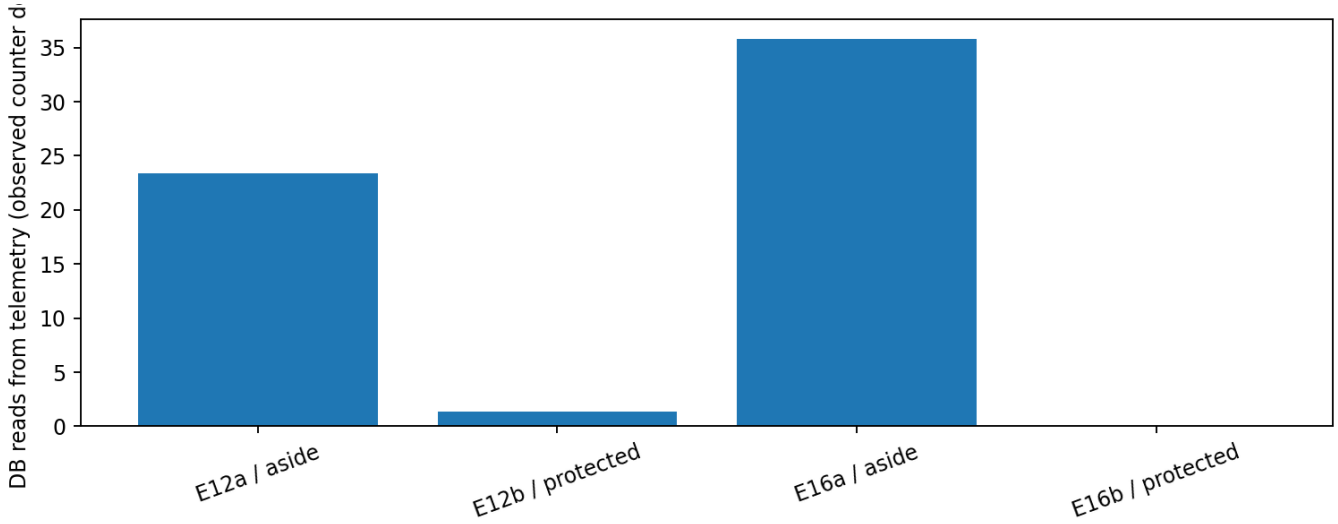


Figure 8. DB-read telemetry around stampede experiments; values are monitoring observations, not exact audited SQL statement counts.

6. Hypotheses and limitations

Hypothesis	Assessment	Comment
H1 latency	Supported for workload	Warm-cache p95 sharply below DB baseline
H2 2x sustainable capacity	Inconclusive	No rate sweep or passing maximum for either mode
H3 DB protection	Supported provisionally	L2 hits ~97%; estimated DB-read reduction ~98%
H4 replica scaling	Supported narrowly	At 150 RPS across 1/2/4 replicas only
H5 fault tolerance	Refuted	High error, dropped arrivals and ~11 s p95
H6 stampede control	Supported provisionally	Fewer DB reads on flush/probe

Hypothesis	Assessment	Comment
H7 invalidation correctness	Supported in sequential test	Concurrent races remain untested
H8 L1/L2 improvement	Refuted	3.10 ms vs 1.51 ms p95; 40% stale

Threats to validity: one execution per scenario, 25-second tests, variable cache temperature, expensive intentionally designed aggregation, no automated rate sweeps, throughput conflated with VU limits/timeouts, mixed database state after writes, and imperfect telemetry deltas. Time-varying resource contention and scenario order could affect rankings. The full profile provides breadth, not repeated statistical evidence.

7. Final architectural recommendation

Adopt conditionally: Redis as a shared cache for expensive, non-authoritative product-demand read projections, with explicit invalidation on mutation, bounded TTL and telemetry. **Revise:** implement circuit breaking, bounded DB fallback, admission control, request coalescing and staged warmup/recovery; investigate why write-aware invalidation stalls under mixed traffic. **Reject for now:** naive aside as a correctness-sensitive read strategy, L1/L2 for mutable cross-replica state, and sharding without proven Redis saturation. **Retain:** PostgreSQL as the transactional source of truth for reservations and payment decisions.

Decision gate for the next campaign: 5 randomized repetitions; arrival-rate sweeps from below DB baseline through cache saturation; 120–180 s steady-state measurements; concurrent read/write revision checks and no overselling; timed Redis fault experiments at loads both below and above DB fallback capacity; 1-second metrics and a measured recovery time. Adopt broadly only after correct behavior, performance SLOs and degraded-mode SLOs all pass.

Data provenance: campaign comparison.csv, manifest.json, hypotheses.json, telemetry JSON, raw k6 summaries/samples and original REPORT.md. All reported values are measured from the uploaded campaign; charts contain no simulated observations.